

Guía de Laboratorio · Sesión 4 · Bóveda inmutable (WORM) · 92-EAN

Parte 1 · La bóveda WORM (MinIO + Object Lock)

Taller Sesión 4 · Ciber-Recuperación (92-EAN) · Duración: 30 min Entorno: **Ubuntu** (VM local, Azure o WSL2) o **macOS**, con **Docker**.

En la Sesión 3 detectaste al actor. Hoy asumes lo peor: **ya está dentro con tus credenciales de administrador** y va por los backups. Vas a construir la copia que sobrevive a eso — y a atacarla tú mismo para comprobarlo.

0. Levanta los servicios del lab

```
cd 04-boveda-inmutable-worm
python3 scripts/generar_dataset.py      # datos sinteticos de
    "produccion" + manifiesto
docker compose up -d                   # MinIO (:9000/:9001) + rest-
    server (:8000)
docker compose ps                       # ambos "running"/"healthy"
```

Instala el cliente `mc` de MinIO (si no lo tienes en el host):

```
# Ubuntu / Linux
curl -sO https://dl.min.io/client/mc/release/linux-amd64/mc && chmod +x
    mc && sudo mv mc /usr/local/bin/
# macOS: brew install minio-mc

mc alias set local http://localhost:9000 labadmin labadmin123
```

Las credenciales `labadmin/labadmin123` son de laboratorio (van en el `docker-compose.yml`). **Nunca** uses credenciales así en producción.

1. Crea dos backups: uno normal y uno blindado (5 min)

El experimento controlado: el **mismo** dato, en dos buckets, con una sola diferencia — el candado.

```
# Bucket NORMAL (como cualquier backup S3 sin proteger)
mc mb local/backup-normal

# BÓVEDA: bucket con Object Lock, retencion COMPLIANCE de 30 días
mc mb --with-lock local/boveda
mc retention set --default COMPLIANCE 30d local/boveda
```

Sube el “backup de anoche” a ambos:

```
tar czf /tmp/backup.tar.gz datasets/caso4/produccion
mc cp /tmp/backup.tar.gz local/backup-normal/
mc cp /tmp/backup.tar.gz local/boveda/

mc ls local/boveda                                # deberia listar
      backup.tar.gz
mc retention info local/boveda/backup.tar.gz # Mode: COMPLIANCE,
      expiring in 29 days
```

Pregunta de diseño (antes de atacar): ¿por qué COMPLIANCE y no GOVERNANCE? GOVERNANCE deja que un rol privilegiado levante el candado — una llave más que el atacante puede robar. COMPLIANCE **no lo levanta nadie** (ni root, ni el proveedor) hasta que expira. Para una bóveda anti-ransomware, esa es la elección.

2. Ataca ambos con tus credenciales válidas (10 min)

Esto es lo que hace el ransomware moderno: no rompe criptografía, **usa las llaves que ya robó**. El script `atacar_boveda.sh` intenta borrar y sobrescribir.

```
# Primero el bucket normal
scripts/atacar_boveda.sh local/backup-normal
```

Verás **[DESTRUIDO] EL BACKUP SE PERDIO**. Sin inmutabilidad, credenciales válidas = backup borrado. Ahora la bóveda:

```
scripts/atacar_boveda.sh local/boveda
```

Verás **[RESISTIO] LA BOVEDA SOBREVIVIO**. El mismo comando, el mismo atacante, dos finales.

3. El matiz que separa a un junior de un senior (10 min)

Corre el borrado “a mano” sobre la bóveda y **lee con atención**:

```
mc rm --recursive --force local/boveda
mc ls local/boveda                # ise ve VACÍO! ¿se perdió?
mc ls --versions local/boveda     # NO: aquí está la verdad
```

Con Object Lock el bucket tiene **versionado**. Tu `rm` no borró nada: solo añadió un **delete marker** encima. La versión bloqueada (`PUT`) sigue intacta debajo. Compáralo con el bucket normal:

```
mc ls --versions local/backup-normal # vacío de verdad – ahí no hay
nada que recuperar
```

Ahora intenta destruir la versión bloqueada **a propósito**, con su version-id:

```
VID=$(mc ls --versions --json local/boveda | \
python3 -c "import json,sys
for l in sys.stdin:
    o=json.loads(l)
    if not o.get('isDeleteMarker') and o.get('size',0)>0:
        print(o['versionId']); break")

mc rm --version-id "$VID" --force local/boveda/backup.tar.gz
```

Respuesta del servidor:

```
mc: <ERROR> ... Object ... is WORM protected and cannot be overwritten
```

Ni siquiera tú, con las llaves maestras, puedes destruirla. Eso es WORM.

4. Recupera desde la bóveda

Quitar el delete marker “resucita” el objeto (o restauras directo por version-id):

```
mc cat --version-id "$VID" local/boveda/backup.tar.gz >
/tmp/recuperado.tar.gz
tar tzf /tmp/recuperado.tar.gz | head      # el backup intacto, listo
para restaurar
```

✅ **Checkpoint Parte 1:** el bucket normal se perdió ante el mismo ataque que la bóveda resistió; entiendes por qué COMPLIANCE > GOVERNANCE y por qué el `rm` en un bucket con lock es reversible. Sigue la Parte 2: proteger el **repositorio de backup** (restic) cuando el candado no está en cada objeto sino en el **servidor**.

Parte 2 · Append-only: el servidor decide, no el cliente (restic)

Taller Sesión 4 · Ciber-Recuperación (92-EAN) · Duración: 25 min

Object Lock protege objetos en almacenamiento S3. Pero muchos backups viven en un **repositorio** (restic, Borg) al que un host de backup accede con una llave. Si ese host cae ante el ransomware, su llave puede **podar la historia** (`forget --prune`). La respuesta: que el **servidor** publique el repo en modo **append-only** — el cliente añade, nunca destruye.

1. Conoce el terreno

En el `docker compose` ya corre `rest-server` (el servidor de repos de restic) publicado con la opción `--append-only` (míralo en `docker-compose.yml`):

```
rest-server:
  environment:
    OPTIONS: "--no-auth --append-only"
```

Instala el cliente `restic` en tu host:

```
# Ubuntu: sudo apt install -y restic      | macOS: brew install
restic                                     restic
restic version
```

Apunta restic al servidor del lab (variables de entorno):

```
export RESTIC_REPOSITORY="rest:http://localhost:8000/"
export RESTIC_PASSWORD="lab123"
restic init                                # inicializa el repositorio
```

2. Respalda producción (el flujo normal, permitido)

```
restic backup datasets/caso4/produccion --host lab
restic snapshots                               # deberia listar 1 snapshot con su
short-id
```

Haz un segundo backup para tener historia (cambia algo primero):

```
echo "nota extra" >>
  datasets/caso4/produccion/ti/runbook_recuperacion.md
restic backup datasets/caso4/produccion --host lab
```

3. El ataque: el cliente comprometido intenta borrar la historia (10 min)

El ransomware en el host de backup haría exactamente esto — quedarse con el snapshot más nuevo (ya cifrado) y **podar** los limpios:

```
SNAP=$(restic snapshots --json | python3 -c "import
    json,sys;print(json.load(sys.stdin)[0]['short_id'])")
restic forget "$SNAP" --prune
```

Respuesta del servidor:

```
Remove(<snapshot/...>) failed: unexpected HTTP response (403): 403
Forbidden
unable to remove snapshot ... from the repository
```

Denegado. El cliente tiene la llave del repo, pero el **servidor** no le permite borrar. La poda legítima la haría *otro* host de confianza, programado, nunca el que respalda producción.

Restic imprime un rastro de error después del **403**. El mensaje que importa es el **403 Forbidden** de la primera línea: ahí rebotó el ataque.

4. Lo que Sí puede hacer el cliente: restaurar (5 min)

Append-only no estorba la recuperación — solo prohíbe destruir:

```
restic restore "$SNAP" --target /tmp/restaurado
ls -R /tmp/restaurado/                # producción restaurada intacta
restic check                          # "no errors were found" – el repo
    está sano
```

5. Compáralo: el mismo ataque en un repo desprotegido (opcional)

Para sentir la diferencia, un repo local **sin** append-only sí deja podar:

```
export RESTIC_REPOSITORY="/tmp/repo-inseguro"
restic init && restic backup datasets/caso4/produccion --host lab
SNAP2=$(restic snapshots --json | python3 -c "import
    json,sys;print(json.load(sys.stdin)[0]['short_id'])")
restic forget "$SNAP2" --prune        # aquí SÍ borra: adiós historial
```

Vuelve a apuntar a la bóveda cuando termines: `export RESTIC_REPOSITORY="rest:http://localhost:8000/"`.

✅ **Checkpoint Parte 2:** el `forget --prune` malicioso rebotó con `403` en el repo append-only, pero backup y restore siguieron funcionando. Entiendes la diferencia entre “la llave hace todo” y “el servidor decide”. Sigue la Parte 3: ¿cómo te enteras de que alguien tocó la bóveda? → **FIM**.

Parte 3 · Vigilar la bóveda (FIM) y restaurar con confianza

Taller Sesión 4 · Ciber-Recuperación (92-EAN) · Duración: 25 min

Ya tienes copias que resisten el borrado. Falta la **cámara de seguridad**: saber si alguien tocó (o intentó tocar) la carpeta del backup, y —cuando restaures— comprobar que lo recuperado es **bit a bit** lo original. Eso cierra el “0” de la regla 3-2-1-1-0: **cero errores al verificar**.

1. AIDE: el inventario notariado (10 min)

AIDE (Advanced Intrusion Detection Environment) guarda una base de datos de hashes de los archivos críticos y avisa de cualquier cambio. Es tu scoring de integridad de la Sesión 2, corriendo como vigilante permanente.

```
# Ubuntu:  sudo apt install -y aide
# macOS:   brew install aide    (o usa el fallback en Python de la
                                seccion 3)
```

Los datos del lab viven en `./data` (volúmenes de MinIO y restic montados al host). Vamos a vigilar esa carpeta. Crea una config mínima:

```
cat > /tmp/aide.conf <<'EOF'
database=file:/tmp/aide.db
database_out=file:/tmp/aide.db.new
gzip_dbout=no
Todos = p+i+n+u+g+s+sha256
./data Todos
EOF

sudo aide -c /tmp/aide.conf --init      # notaría inicial (crea la
base)
sudo mv /tmp/aide.db.new /tmp/aide.db  # promueve la base recién
creada
```

La base de datos de AIDE (`aide.db`) es tan valiosa como el backup: guárdala **en la bóveda** (Parte 1). Si el atacante puede reescribir la notaría, el FIM no vale nada.

2. Simula la manipulación y detéctala (10 min)

Un atacante toca la carpeta de backup — añade un archivo y modifica otro:

```
echo "carga_util_ransomware" | tee data/minio/nota_atacante.txt
    >/dev/null
# (si tienes un objeto extraible, modifícalo; si no, basta el archivo
    nuevo)

sudo aide -c /tmp/aide.conf --check
```

AIDE reporta el cambio:

```
Added entries:
f+++++: /.../data/minio/nota_atacante.txt
AIDE found differences between database and filesystem!!
```

Ahí tienes la alerta: **algo cambió en la bóveda que tú no autorizaste**. En producción, esto es lo que **Wazuh FIM** hace en tiempo real y escala a cientos de hosts (mismo concepto, agente que vigila **syscheck** y alerta al SIEM). AIDE es la versión que ves “por dentro” en un solo host.

Sin AIDE instalado (fallback): el script **verificar_integridad.py** aplica la misma lógica de hashes sobre una carpeta contra el manifiesto del dataset.

3. Restaura desde la bóveda y verifica el “0” (5 min)

La detección disparó → ahora recuperas desde la copia inmutable de la Parte 1 (o el repo append-only de la Parte 2) y **compruebas la integridad**:

```
# recupera (ejemplo con restic de la Parte 2)
export RESTIC_REPOSITORY="rest:http://localhost:8000/"
RESTIC_PASSWORD="lab123"
SNAP=$(restic snapshots --json | python3 -c "import
    json,sys;print(json.load(sys.stdin)[-1]['short_id'])")
restic restore "$SNAP" --target /tmp/recuperado

# verifica bit a bit contra el manifiesto SHA-256 (scoring de la Sesión
    2)
python3 scripts/verificar_integridad.py /tmp/recuperado/prod
```

Salida esperada:

```
7/7 intactos · 0 alterados · 0 faltantes
RESULTADO: restauracion 100% integra. Cero errores de verificacion.
```

Ese **0 alterados · 0 faltantes** es el **“0” de 3-2-1-1-0**: un backup no está probado hasta que restauras y verificas. Una copia sin esta prueba es una promesa, no un respaldo.

4. Cierre de la sesión y del ciclo técnico

El arsenal completo que te llevas:

```
Deteccion (S3) → la regla dispara: "borraron las shadow copies"
|
▼
Bodega WORM (Parte 1)    el backup EXISTE y no se puede destruir
Append-only (Parte 2)    el cliente comprometido no poda la historia
FIM / AIDE (Parte 3)     sabes que alguien toco la bodega
Verificacion (S2)         restauras y confirmas: 0 errores → no
pagas rescate
```

- **Object Lock / append-only** aportan **inmutabilidad** (la copia sobrevive).
- **AIDE / Wazuh** aportan **visibilidad** (te enteras del intento).
- **La verificación de hashes** aporta **confianza** (lo restaurado es correcto).

Sin las tres, la recuperación es un acto de fe. Esta es la base de la Sesión 5: cuando todo esto dispara a las 3 a.m., un **copiloto RAG local** te guía por el runbook — sin sacar tus procedimientos de la sala.

✅ **Checkpoint Parte 3:** AIDE detectó la manipulación de la carpeta de backup, restauraste desde la bóveda y `verificar_integridad.py` confirmó **0 alterados** • **0 faltantes**. Cerraste la regla 3-2-1-1-0 de punta a punta.

Limpieza

```
docker compose down          # detiene MinIO y rest-server (conserva
    ./data)
docker compose down -v       # ...y borra tambien los volúmenes del
    lab
```